

UNIVERSIDAD EAFIT

Escuela de Ingeniería

Ingeniería de Sistemas

Informe Técnico

Práctica 2 — Construcción de la Plataforma Hack:
Lenguaje Máquina, CPU y Computador

Organización de Computadores 2026-2

Docente: José Luis Montoya Pareja

Equipo Gentlemen

Cristóbal Machado Sánchez

Mathías Vélez Londoño

Sebastián Sánchez Gómez

Medellín, Colombia

September 19, 2026

Contents

1	Introducción	2
2	Proyecto 4 — Programación en Ensamblador	2
2.1	Mult.asm	2
2.2	Fill.asm	2
2.3	SumN.asm (programa adicional del curso)	3
2.4	CopyBlock.asm (programa adicional del curso)	3
3	Proyecto 5 — Componentes de Hardware	3
3.1	Memory.hdl	3
3.2	CPU.hdl	5
3.3	Computer.hdl	7
4	Dificultades Encontradas y Decisiones de Diseño	7
5	Conclusiones	8

1 Introducción

Este documento presenta el desarrollo de la Práctica 2 del curso de Organización de Computadores, correspondiente a los Proyectos 4 y 5 de la plataforma Nand2Tetris. El objetivo de esta etapa es conectar los componentes de hardware construidos previamente (compuertas lógicas, ALU y memoria RAM) con el nivel de software, mediante el desarrollo de programas en lenguaje ensamblador Hack y la construcción de los componentes que permiten a un computador ejecutar dichos programas: la unidad de memoria (**Memory**), la unidad central de procesamiento (**CPU**) y la integración completa del computador (**Computer**).

El informe describe, para cada componente, la lógica de diseño empleada, las decisiones técnicas tomadas y la evidencia de verificación mediante los simuladores oficiales de Nand2Tetris.

Distribución del trabajo

- **Cristóbal Machado Sánchez:** implementación de `Memory.hdl` y colaboración en `Computer.hdl`.
- **Mathías Vélez Londoño:** implementación de `CPU.hdl` y colaboración en `Computer.hdl`.
- **Sebastián Sánchez Gómez:** desarrollo de los programas en ensamblador del Proyecto 4.

2 Proyecto 4 — Programación en Ensamblador

El lenguaje ensamblador Hack cuenta con dos tipos de instrucciones de 16 bits: la **instrucción A** (carga un valor o dirección en el registro A) y la **instrucción C** (ejecuta una operación de la ALU y opcionalmente almacena el resultado y/o ejecuta un salto condicional). A continuación se describe la lógica empleada en cada uno de los cuatro programas desarrollados.

2.1 Mult.asm

Objetivo: calcular el producto de dos números almacenados en `RAM[0]` y `RAM[1]`, guardando el resultado en `RAM[2]`.

Lógica: dado que la plataforma Hack no cuenta con una instrucción de multiplicación nativa, el producto se calcula mediante sumas repetidas: se inicializa un acumulador en cero y se suma el valor de `RAM[0]` tantas veces como indique `RAM[1]`, utilizando un contador que se decrementa en cada iteración y una instrucción de salto condicional (`JGT` o similar) que mantiene el ciclo activo mientras el contador sea mayor que cero.

2.2 Fill.asm

Objetivo: ennegrecer toda la pantalla mientras una tecla esté presionada, y limpiarla (dejarla en blanco) cuando no lo esté.

Lógica: el programa consulta continuamente la dirección de memoria del teclado (`RAM[24576]`). Si el valor leído es distinto de cero, recorre todo el mapa de memoria de pantalla (direcciones 16384 a 24575) escribiendo `-1` (todos los bits en 1, es decir, negro) en cada palabra; si el valor es cero, recorre el mismo rango escribiendo `0` (blanco). Este proceso se repite indefinidamente en un ciclo principal (bucle infinito), característico de los programas de control de hardware.

2.3 SumN.asm (programa adicional del curso)

Objetivo: calcular la suma de los primeros N números naturales, donde N se recibe en `RAM[0]` y el resultado se almacena en `RAM[1]`.

Lógica: se emplea un ciclo iterativo controlado por un contador que inicia en N y decrece en una unidad en cada vuelta; en cada iteración se acumula su valor sobre un total, y el ciclo finaliza cuando el contador llega a cero, momento en el que el acumulador se transfiere a `RAM[1]`. Por ejemplo, para `RAM[0] = 5`, el resultado esperado es `RAM[1] = 15` ($1 + 2 + 3 + 4 + 5$).

2.4 CopyBlock.asm (programa adicional del curso)

Objetivo: copiar un bloque de N posiciones de memoria desde una dirección de origen hacia una dirección de destino, donde origen, destino y N se reciben en `RAM[0]`, `RAM[1]` y `RAM[2]` respectivamente.

Lógica: el programa utiliza direccionamiento indirecto: en cada iteración calcula la dirección efectiva de lectura (origen + índice) y de escritura (destino + índice) sumando un contador de avance a las direcciones base, lee el valor en la posición de origen y lo escribe en la posición de destino correspondiente, incrementando el índice hasta haber copiado las N posiciones solicitadas. Este patrón demuestra el manejo de direccionamiento indirecto mediante el registro A como puntero.

3 Proyecto 5 — Componentes de Hardware

Esta sección documenta los tres componentes de hardware construidos en HDL: **Memory**, **CPU** y **Computer**, que en conjunto conforman la plataforma Hack capaz de ejecutar programas en lenguaje máquina.

3.1 Memory.hdl

Función: **Memory** unifica en una sola interfaz el acceso a los tres espacios de direcciones del computador Hack: la memoria de datos (RAM), la memoria mapeada a la pantalla (Screen) y la memoria mapeada al teclado (Keyboard). La CPU no necesita conocer a cuál de estos tres dispositivos pertenece una dirección: **Memory** recibe la dirección de 15 bits y enruta internamente la operación de lectura o escritura hacia el componente correcto.

Mapa de memoria

Rango	Dispositivo	Tamaño	Bit(s) decisivo(s)
0 – 16383	RAM	$16384 = 2^{14}$	<code>address[14] = 0</code>
16384 – 24575	Screen	$8192 = 2^{13}$	<code>address[14]=1, address[13]=0</code>
24576	Keyboard	1	<code>address[14]=1, address[13]=1</code>

Código implementado

```
CHIP Memory {
    IN in[16], load, address[15];
    OUT out[16];

    PARTS:
        DMux(in=load, sel=address[14], a=loadRAM, b=loadScreenOrKbd);
        DMux(in=loadScreenOrKbd, sel=address[13], a=loadScreen, b=loadKbd);
}
```

```

RAM16K(in=in, load=loadRAM, address=address[0..13], out=ramOut);
Screen(in=in, load=loadScreen, address=address[0..12], out=screenOut);
Keyboard(out=kbdOut);

Mux16(a=screenOut, b=kbdOut, sel=address[13], out=screenOrKbdOut);
Mux16(a=ramOut, b=screenOrKbdOut, sel=address[14], out=out);
}

```

Explicación línea por línea

- **Primer DMux:** reparte la señal de escritura (`load`) según `address[14]`. Si es 0 (zona RAM), `loadRAM` recibe la señal y `loadScreenOrKbd` permanece en 0; si es 1, ocurre lo inverso. Esto garantiza que solo el dispositivo correspondiente a la dirección reciba la orden de escritura.
- **Segundo DMux:** dentro de la zona alta ya seleccionada, vuelve a repartir la señal según `address[13]` para distinguir entre Screen y Keyboard.
- **RAM16K:** recibe `loadRAM` y únicamente los 14 bits menos significativos de la dirección (`address[0..13]`), ya que el bit 14 se utilizó para decidir la zona y RAM16K solo necesita 14 bits para direccionar sus 16384 posiciones internas.
- **Screen:** recibe `loadScreen` y los 13 bits menos significativos (`address[0..12]`), suficientes para direccionar sus 8192 posiciones.
- **Keyboard:** no posee entradas de datos (`in`), carga (`load`) ni dirección (`address`), ya que es un dispositivo exclusivamente de lectura: su valor lo determina el estado físico del teclado, no una operación de escritura de la CPU.
- **Los dos Mux16 finales:** realizan el proceso inverso al de escritura para seleccionar la salida de lectura. El primero elige entre la salida de Screen y de Keyboard según `address[13]`; el segundo elige entre RAM y el resultado anterior según `address[14]`, produciendo la salida final `out`.

Verificación

El chip fue cargado en el Hardware Simulator de Nand2Tetris junto con el script de prueba oficial `Memory.tst`, obteniendo el resultado “**Comparison ended successfully**”. La prueba incluye verificación de lectura y escritura en RAM y Screen, así como una prueba interactiva de Keyboard que requiere la pulsación de una tecla física durante la ejecución del script.

3.2 CPU.hdl

Función: la CPU es la unidad encargada de decodificar y ejecutar cada instrucción de 16 bits recibida desde la memoria de instrucciones (ROM), coordinando la ALU, los registros internos A y D, y el contador de programa (PC) para determinar qué operación realizar, dónde almacenar su resultado y si corresponde ejecutar un salto.

Formato de instrucciones

Tipo	Formato de bits	Significado
Instrucción A	0 vvvvvvvvvvvvvvv	Carga el valor v (15 bits) en el registro A
Instrucción C	111a c1c2c3c4c5c6 d1d2d3 j1j2j3	a: elige A o M; c1-c6: control de la ALU; d1-d3: destino (A, D, M); j1-j3: condición de salto

Código implementado

```
CHIP CPU {
  IN  inM[16], instruction[16], reset;
  OUT outM[16], writeM, addressM[15], pc[15];

  PARTS:
  Mux16(a=instruction, b=aluOut, sel=instruction[15], out=aInput);

  Not(in=instruction[15], out=isAInstruction);
  And(a=instruction[15], b=instruction[5], out=isCandDestA);
  Or(a=isAInstruction, b=isCandDestA, out=loadA);
  ARegister(in=aInput, load=loadA, out=aOut, out[0..14]=addressM);

  Mux16(a=aOut, b=inM, sel=instruction[12], out=aOrM);

  And(a=instruction[15], b=instruction[4], out=loadD);
  DRegister(in=aluOut, load=loadD, out=dOut);

  ALU(x=dOut, y=aOrM,
      zx=instruction[11], nx=instruction[10],
      zy=instruction[9], ny=instruction[8],
      f=instruction[7], no=instruction[6],
      out=aluOut, out=outM, zr=zr, ng=ng);

  And(a=instruction[15], b=instruction[3], out=writeM);

  Not(in=ng, out=notNg);
  Not(in=zr, out=notZr);
  And(a=notNg, b=notZr, out=positive);

  And(a=instruction[2], b=ng, out=jumpNeg);
  And(a=instruction[1], b=zr, out=jumpZero);
  And(a=instruction[0], b=positive, out=jumpPos);

  Or(a=jumpNeg, b=jumpZero, out=jumpNegOrZero);
  Or(a=jumpNegOrZero, b=jumpPos, out=jumpCond);
```

```

    And(a=jumpCond, b=instruction[15], out=jump);

    PC(in=aOut, load=jump, inc=true, reset=reset, out[0..14]=pc);
}

```

Explicación línea por línea

- **Mux16 inicial:** decide qué valor entra al registro A. Si la instrucción es de tipo A (`instruction[15]=0`), entra la instrucción misma (el valor a cargar); si es de tipo C, entra el resultado de la ALU (para instrucciones como $A=D-1$).
- **Cálculo de loadA:** el registro A debe cargarse en dos casos: siempre que la instrucción sea de tipo A (`isAInstruction`), o cuando sea de tipo C y su bit de destino d_1 (`instruction[5]`) indique que el resultado debe guardarse en A. Ambos casos se combinan con un `Or`.
- **ARegister:** almacena el valor de A; sus 15 bits inferiores se exponen directamente como `addressM`, la dirección de memoria que la CPU expone hacia `Memory`.
- **Mux16 A/M:** según el bit a (`instruction[12]`), la ALU opera con el valor de A o con el valor leído de memoria (`inM`), permitiendo instrucciones como $D=D+M$.
- **loadD:** el registro D solo se carga cuando la instrucción es de tipo C y su bit de destino d_2 (`instruction[4]`) lo indica.
- **ALU:** recibe D y (A o M) como operandos, junto con los seis bits de control extraídos directamente de la instrucción (`instruction[6..11]`), reutilizando la ALU construida en el Proyecto 2.
- **writeM:** se activa únicamente si la instrucción es de tipo C y su bit de destino d_3 (`instruction[3]`) lo indica, señalando a `Memory` que debe escribirse el resultado en la dirección `addressM`.
- **Lógica de salto:** se evalúan tres condiciones — salto si el resultado es negativo (`ng` y $j_1 = \text{instruction}[2]$), si es cero (`zr` y $j_2 = \text{instruction}[1]$), o si es positivo (ni negativo ni cero, junto con $j_3 = \text{instruction}[0]$) — combinadas con `Or`. La señal final `jump` solo se activa si además la instrucción es de tipo C.
- **PC:** si `jump` está activo, carga el valor de A (nueva dirección de salto); en caso contrario, incrementa en uno (`inc=true`) para avanzar a la siguiente instrucción; si `reset` está activo, reinicia en cero independientemente de las demás señales.

Verificación

El chip fue verificado con el script oficial `CPU.tst`, obteniendo “**Comparison ended successfully**”. Las pruebas cubren instrucciones A, instrucciones C con distintas combinaciones de destino (A, D, M, y combinaciones AD, MD, AMD), operaciones aritméticas y lógicas de la ALU, y saltos condicionales bajo las tres condiciones posibles (negativo, cero, positivo).

3.3 Computer.hdl

Función: `Computer` integra los tres grandes bloques de la arquitectura Hack en un único computador funcional: la memoria de instrucciones (`ROM32K`, de solo lectura), la CPU y la unidad de `Memory`, conectando sus señales para formar el ciclo de ejecución completo: obtener instrucción, ejecutarla, actualizar memoria y avanzar el contador de programa.

Código implementado

```
CHIP Computer {  
  
    IN reset;  
  
    PARTS:  
    ROM32K(address=pc, out=instruction);  
  
    CPU(inM=memOut, instruction=instruction, reset=reset,  
        outM=cpuOut, writeM=writeM, addressM=addressM, pc=pc);  
  
    Memory(in=cpuOut, load=writeM, address=addressM, out=memOut);  
}
```

Explicación línea por línea

- **ROM32K:** almacena el programa a ejecutar (cargado externamente por el simulador). Su dirección de lectura es el valor actual del contador de programa (`pc`), y su salida es la instrucción vigente.
- **CPU:** recibe la instrucción leída de la ROM, el valor actual leído de memoria (`inM=memOut`, retroalimentación del ciclo anterior) y la señal de `reset`. Produce el resultado a escribir en memoria (`outM`), la señal de si se debe escribir (`writeM`), la dirección de memoria involucrada (`addressM`) y el nuevo valor del contador de programa (`pc`), que a su vez alimenta a `ROM32K` para la siguiente instrucción.
- **Memory:** recibe el valor a escribir (`cpuOut`), la señal de escritura (`writeM`) y la dirección (`addressM`) provenientes de la CPU, y expone su salida (`memOut`) de vuelta hacia la entrada `inM` de la CPU, cerrando así el ciclo de retroalimentación entre procesador y memoria.

Verificación

El chip fue verificado con los tres scripts oficiales disponibles: **`ComputerAdd.tst`** (suma de dos constantes), **`ComputerMax.tst`** (cálculo del máximo entre dos valores) y **`ComputerRect.tst`** (dibujo de un rectángulo en pantalla). Los tres scripts finalizaron con “**Comparison ended successfully**”, confirmando que la integración de ROM, CPU y Memory reproduce correctamente el comportamiento esperado de un computador Hack completo.

4 Dificultades Encontradas y Decisiones de Diseño

- **Organización de rutas de archivos:** en proyectos anteriores del curso se identificó que Nand2Tetris organiza los stubs oficiales en subcarpetas específicas dentro de cada proyecto (por ejemplo, `a/` y `b/` en el Proyecto 3), lo que requirió verificar cuidadosamente la ubicación real de cada archivo antes de implementar la lógica, evitando así editar copias que el simulador no reconocía.

- **Dependencias entre chips contruidos por distintos integrantes:** al trabajar en equipo, se presentaron casos en que un chip dependía de otro implementado por un compañero en una máquina distinta; la solución consistió en sincronizar el repositorio compartido (`git pull`) y copiar los archivos ya validados hacia la carpeta local de trabajo antes de continuar.
- **Control de versión de línea (CRLF/LF):** se detectó en una ocasión que un archivo guardado con terminaciones de línea estilo Windows (CRLF) provocaba fallos de comparación silenciosos en el simulador; se solucionó normalizando el archivo a formato Unix (LF).
- **Diseño jerárquico de Memory:** se decidió reutilizar el patrón de selección en cascada (DMux/Mux) ya empleado en los proyectos de memoria RAM, aplicándolo esta vez para distinguir entre tres dispositivos de memoria en lugar de múltiples bloques de un mismo tipo.
- **Verificación incremental:** cada componente (**Memory**, **CPU**, **Computer**) se probó de forma independiente con su script oficial antes de integrarlo con los demás, lo que permitió aislar y corregir errores de forma más eficiente que si se hubiera probado únicamente el sistema completo.

5 Conclusiones

El desarrollo de esta práctica permitió comprender de manera práctica la relación entre el software (programas en lenguaje ensamblador) y el hardware (CPU, memoria y su integración) en una arquitectura de computador real. La construcción de **Memory** evidenció cómo un espacio de direcciones puede unificar dispositivos de naturaleza distinta (memoria de datos, pantalla y teclado) bajo una interfaz común; la construcción de la CPU permitió comprender el proceso de decodificación de instrucciones y el papel de cada bit de control; y la integración en **Computer** consolidó la comprensión del ciclo completo de ejecución de un programa: búsqueda de instrucción, decodificación, ejecución y actualización de estado.

Todos los componentes de hardware fueron verificados exitosamente mediante los scripts de prueba oficiales de Nand2Tetris, y los programas en ensamblador fueron diseñados siguiendo los patrones estándar de control de flujo (ciclos, condicionales y direccionamiento indirecto) propios del lenguaje máquina Hack.